

Python Programming for Beginners

Part 5 - Loops in Python

Topics Covered

- `for` Loop
- `while` Loop
- `range()` Function
- Nested Loops
- `break`
- `continue`
- `pass`
- Loop else Clause

Learning Objectives

By the end of this lesson, you will be able to:

- Understand what loops are.
- Explain why loops are used.
- Write programs using the `for` loop.
- Use the `range()` function.
- Write programs using the `while` loop.
- Control loops using `break`, `continue`, and `pass`.
- Write nested loops.
- Use the `else` clause with loops.

What are Loops?

A **loop** is a control structure that allows a block of code to be executed repeatedly until a condition is met or a sequence has been processed.

Instead of writing the same code multiple times, a loop automates repetitive tasks.

Example

Without Loop

```
In [ ]: print("Hello")
        print("Hello")
```

```
print("Hello")
print("Hello")
print("Hello")
```

With Loop

```
In [ ]: for i in range(5):
        print("Hello")
```

Both programs produce the same output, but the loop requires much less code.

Why Do We Need Loops?

Loops help us repeat a block of code efficiently.

They are used to:

- Reduce repetitive code.
- Process collections such as lists and strings.
- Perform repetitive calculations.
- Read data repeatedly.
- Improve code readability and efficiency.

Without loops, programs become longer and harder to maintain.

Types of Loops

Python provides two main types of loops.

Loop	Description
for Loop	Repeats code over a sequence or a fixed number of times
while Loop	Repeats code while a condition is True

for Loop

The **for loop** is used to execute a block of code repeatedly by iterating over a sequence, such as a **string, list, tuple, set, dictionary**, or the values generated by the **range()** function.

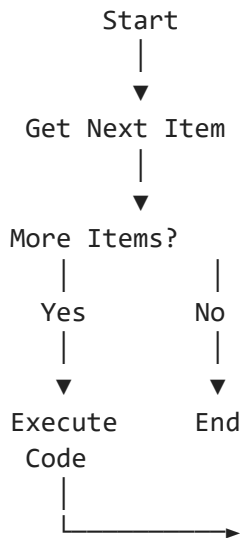
It is commonly used when you know how many times you want to repeat a task or when you want to process each item in a sequence.

Syntax

```
for variable in sequence:  
    # Code to execute
```

Note: The code inside the `for` loop must be properly indented.

Flow Diagram



Example 1: Print Numbers

```
In [ ]: for i in range(1, 6):  
        print(i)
```

Output

```
1  
2  
3  
4  
5
```

Explanation

The loop starts from `1` and ends before `6`, printing one number in each iteration.

Example 2: Print a Message Multiple Times

```
In [ ]: for i in range(3):  
        print("Welcome to Python")
```

Output

```
Welcome to Python  
Welcome to Python  
Welcome to Python
```

Explanation

The loop executes **3 times**, printing the same message in each iteration.

Example 3: Iterate Through a String

```
In [ ]: for letter in "Python":  
        print(letter)
```

Output

P
y
t
h
o
n

Explanation

The loop visits each character in the string one by one.

Example 4: Iterate Through a List

```
In [ ]: fruits = ["Apple", "Banana", "Mango"]  
  
for fruit in fruits:  
    print(fruit)
```

Output

Apple
Banana
Mango

Explanation

The loop accesses each item in the list and prints it.

Key Points

- The `for` loop is used to repeat a block of code.
- It iterates over sequences such as strings, lists, tuples, sets, dictionaries, and `range()`.
- The loop variable stores one item from the sequence during each iteration.
- The loop ends automatically after processing all items.
- Proper indentation is required inside the loop.

`range()` Function

The `range()` function is used to generate a sequence of numbers. It is commonly used with the `for` loop to repeat a block of code a specific number of times.

The `range()` function does **not** create a list. Instead, it returns a **range object**, which generates numbers one by one.

Syntax

1. `range(stop)`

`range(stop)`

Generates numbers from **0** to **stop - 1**.

Example

```
In [ ]: for i in range(5):  
        print(i)
```

Output

```
0  
1  
2  
3  
4
```

Explanation

The loop starts at **0** by default and stops before **5**.

Syntax

2. `range(start, stop)`

`range(start, stop)`

Generates numbers from **start** to **stop - 1**.

Example

```
In [ ]: for i in range(2, 7):  
        print(i)
```

Output

```
2  
3  
4
```

5
6

Explanation

The loop starts at **2** and stops before **7**.

Syntax

3. `range(start, stop, step)`

Output

2
3
4
5
6

Explanation

The loop starts at **2** and stops before **7**.

Syntax

3. `range(start, stop, step)`

`range(start, stop, step)`

Generates numbers from **start** to **stop - 1**, increasing or decreasing by **step**.

Example 1: Positive Step

```
In [ ]: for i in range(1, 11, 2):  
        print(i)
```

Output

1
3
5
7
9

Explanation

The loop starts at **1**, increases by **2**, and stops before **11**.

Example 2: Negative Step

```
In [ ]: for i in range(10, 0, -2):
        print(i)
```

Output

10
8
6
4
2

Explanation

The loop starts at **10**, decreases by **2**, and stops before **0**.

Summary Table

Function	Description	Example
<code>range(stop)</code>	Starts from <code>0</code> and stops before <code>stop</code>	<code>range(5)</code>
<code>range(start, stop)</code>	Starts from <code>start</code> and stops before <code>stop</code>	<code>range(2, 7)</code>
<code>range(start, stop, step)</code>	Starts from <code>start</code> , changes by <code>step</code> , and stops before <code>stop</code>	<code>range(1, 10, 2)</code>

Key Points

- `range()` is commonly used with the `for` loop.
- The **stop value is not included** in the sequence.
- The default start value is `0`.
- The default step value is `1`.
- A negative step creates a decreasing sequence.
- `range()` returns a **range object**, not a list.

Nested for Loop

A **Nested for Loop** is a `for` loop placed **inside another for loop**.

The **outer loop** executes first. For each iteration of the outer loop, the **inner loop** executes completely.

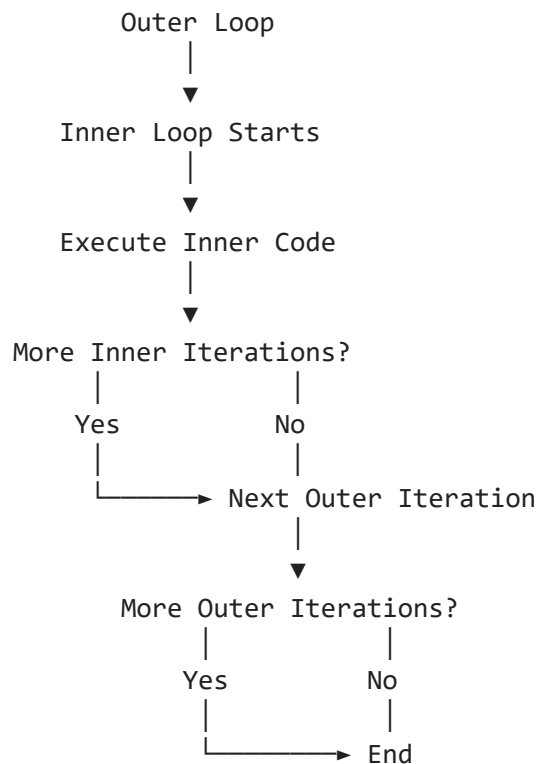
Nested loops are commonly used to work with **patterns, tables, matrices**, and **2D data**.

Syntax

```
for outer_variable in sequence:
    for inner_variable in sequence:
        # Code to execute
```

Note: The inner loop runs completely for each iteration of the outer loop.

Flow Diagram



Example 1: Print Coordinates

```
In [ ]: for i in range(1, 4):
        for j in range(1, 4):
            print(i, j)
```

Output

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

Explanation

- The outer loop runs **3 times**.
- For each outer loop iteration, the inner loop also runs **3 times**.

- Total iterations = $3 \times 3 = 9$.
-

Example 2: Print a Square Pattern

```
In [ ]: for i in range(3):
        for j in range(3):
            print("*", end=" ")
        print()
```

Output

```
* * *
* * *
* * *
```

Explanation

The inner loop prints three stars in one row, and the outer loop repeats this process three times.

Example 3: Multiplication Table (1 to 3)

```
In [ ]: for i in range(1, 4):
        for j in range(1, 4):
            print(i * j, end=" ")
        print()
```

Output

```
1 2 3
2 4 6
3 6 9
```

Explanation

Each value is calculated by multiplying the outer loop variable (`i`) by the inner loop variable (`j`).

Key Points

- A nested `for` loop is a loop inside another `for` loop.
- The inner loop completes all its iterations before the outer loop moves to the next iteration.
- Nested loops are useful for printing patterns, multiplication tables, and processing two-dimensional data.
- Total iterations = **Outer Loop Iterations × Inner Loop Iterations**.
- Proper indentation is required for both loops.

while Loop

The **while** loop repeatedly executes a block of code **as long as a specified condition is True**.

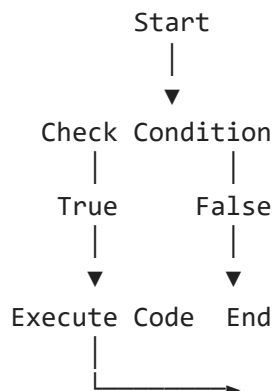
Unlike the **for** loop, which is used when the number of iterations is known, the **while** loop is useful when the number of iterations is **not known in advance**.

Syntax

```
while condition:  
    # Code to execute
```

Note: The code inside the **while** loop must be properly indented.

Flow Diagram



Example 1: Print Numbers from 1 to 5

```
In [ ]: count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

Output

```
1  
2  
3  
4  
5
```

Explanation

- The variable **count** starts at **1**.

- The loop runs while `count <= 5` .
 - After each iteration, `count` is increased by `1` .
 - When `count` becomes `6` , the condition is `False` , and the loop stops.
-

Example 2: Print a Message Three Times

```
In [ ]: i = 1

while i <= 3:
    print("Welcome to Python")
    i += 1
```

Output

```
Welcome to Python
Welcome to Python
Welcome to Python
```

Example 3: Countdown

```
In [ ]: num = 5

while num > 0:
    print(num)
    num -= 1

print("Done!")
```

Output

```
5
4
3
2
1
Done!
```

Explanation

The loop prints numbers from `5` to `1` . After reaching `0` , the condition becomes `False` , and `"Done!"` is printed.

for Loop vs while Loop

for Loop	while Loop
Used when the number of iterations is known.	Used when the number of iterations is unknown.
Iterates over a sequence or <code>range()</code> .	Repeats while a condition is <code>True</code> .

for Loop

Stops automatically after the sequence ends.

while Loop

Stops only when the condition becomes `False`.

Key Points

- The `while` loop repeats code while a condition is `True`.
- The condition is checked before each iteration.
- Update the loop variable inside the loop to avoid an infinite loop.
- Use a `while` loop when the number of iterations is not known beforehand.
- Proper indentation is required inside the loop.

Infinite Loop

An **Infinite Loop** is a loop that **never ends** because its condition always remains `True`.

Infinite loops continue running until they are stopped manually or interrupted by the program.

Warning: Be careful when writing `while` loops. Always make sure the loop condition eventually becomes `False`.

Syntax

```
while True:
    # Code to execute
```

Example 1: Infinite Loop

```
In [ ]: while True:
        print("Hello")
```

Output

```
Hello
Hello
Hello
...
```

Explanation

The condition `True` is always true, so the loop never stops.

Example 2: Infinite Loop Caused by a Missing Update

```
In [ ]: count = 1

while count <= 5:
    print(count)
```

Output

```
1
1
1
1
...
```

Explanation

The value of `count` is never updated, so the condition `count <= 5` always remains `True`, resulting in an infinite loop.

Example 3: Correct Way

```
In [ ]: count = 1

while count <= 5:
    print(count)
    count += 1
```

Output

```
1
2
3
4
5
```

Explanation

The value of `count` increases after each iteration. When `count` becomes `6`, the condition becomes `False`, and the loop stops.

How to Stop an Infinite Loop?

- Update the loop variable so the condition eventually becomes `False`.
 - Use the `break` statement when a specific condition is met.
 - Press **Ctrl + C** in the terminal to stop a running program manually.
-

Key Points

- An infinite loop never ends because its condition always remains `True` .
- It often occurs when the loop variable is not updated.
- Use infinite loops carefully in programs such as menus, games, and servers.
- Use `break` or update the loop condition to exit the loop.

break Statement

The `break` statement is used to **immediately terminate a loop**. When Python encounters a `break` statement, it exits the current loop, and the program continues with the next statement after the loop.

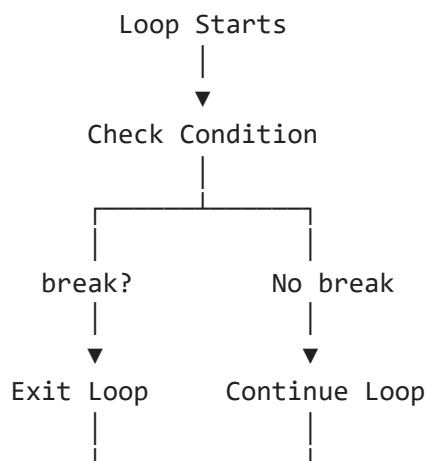
The `break` statement can be used with both `for` and `while` loops.

Syntax

`break`

Note: The `break` statement is usually used inside an `if` statement.

Flow Diagram



Example 1: break with for Loop

```
In [ ]: for i in range(1, 6):  
        if i == 4:  
            break  
        print(i)
```

Output

```
1  
2
```

3

Explanation

When `i` becomes `4`, the `break` statement is executed, and the loop stops immediately.

Example 2: `break` with `while` Loop

```
In [ ]: count = 1

while count <= 5:
    if count == 4:
        break
    print(count)
    count += 1
```

Output

1
2
3

Explanation

The loop ends as soon as `count` becomes `4`.

Example 3: Stop an Infinite Loop

```
count = 1
```

```
while True: print(count)
```

```
    if count == 5:
        break
```

```
    count += 1
```

Output

1
2
3
4
5

Explanation

The loop starts as an infinite loop (`while True`), but the `break` statement stops it when `count` reaches `5`.

Key Points

- The `break` statement immediately exits the current loop.
- It can be used with both `for` and `while` loops.
- The code after the loop continues to execute.
- `break` is commonly used with an `if` statement.
- It is useful for stopping infinite loops or ending a loop when a specific condition is met.

continue Statement

The `continue` statement is used to **skip the current iteration** of a loop and move directly to the next iteration.

Unlike the `break` statement, `continue` **does not terminate the loop**. It only skips the remaining code in the current iteration.

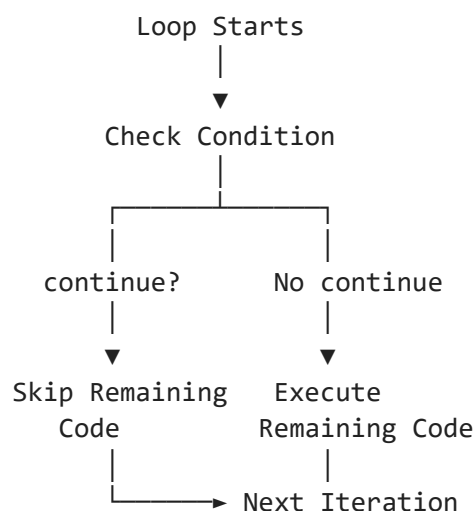
The `continue` statement can be used with both `for` and `while` loops.

Syntax

`continue`

Note: The `continue` statement is usually used inside an `if` statement.

Flow Diagram



Example 1: `continue` with `for` Loop

```
In [ ]: for i in range(1, 6):  
        if i == 3:
```



```
        continue
    print(i)
```

Output

```
1
2
4
5
```

Explanation

When `i` becomes `3`, the `continue` statement skips the `print()` statement and moves to the next iteration.

Example 2: `continue` with `while` Loop

```
In [ ]: count = 0

while count < 5:
    count += 1

    if count == 3:
        continue

    print(count)
```

Output

```
1
2
4
5
```

Explanation

When `count` becomes `3`, the `continue` statement skips the `print()` statement and the loop continues with the next iteration.

Example 3: Print Only Odd Numbers

```
In [ ]: for i in range(1, 11):
        if i % 2 == 0:
            continue
        print(i)
```

Output

```
1
3
5
7
9
```

Explanation

Even numbers are skipped using the `continue` statement, so only odd numbers are printed.

break vs continue

break	continue
Terminates the loop immediately.	Skips the current iteration only.
Exits the loop completely.	Continues with the next iteration.
Used when the loop should stop.	Used when only one iteration should be skipped.

Key Points

- The `continue` statement skips the current iteration of a loop.
- It does **not** terminate the loop.
- It can be used with both `for` and `while` loops.
- It is commonly used with an `if` statement.
- `continue` is useful when you want to ignore specific values while continuing the loop.

pass Statement

The `pass` statement is a **null statement** in Python. It does nothing when executed.

It is used as a **placeholder** when a statement is required syntactically, but you do not want to write any code yet.

The `pass` statement is commonly used with loops, functions, classes, and conditional statements during program development.

Syntax

`pass`

Example 1: pass with for Loop

```
In [ ]: for i in range(1, 6):  
        if i == 3:
```

```
pass  
print(i)
```

Output

```
1  
2  
3  
4  
5
```

Explanation

When `i` is `3`, the `pass` statement is executed. Since `pass` does nothing, the program continues normally and prints all the numbers.

Example 2: `pass` with `if` Statement

```
In [ ]: number = 10  
  
if number > 0:  
    pass  
  
print("Program continues...")
```

Output

```
Program continues...
```

Explanation

The `pass` statement acts as a placeholder. It allows the program to run without producing an error.

Example 3: `pass` with `while` Loop

```
count = 1
```

```
while count <= 3: pass print(count) count += 1
```

Output

```
1  
2  
3
```

Explanation

The `pass` statement has no effect. The loop executes normally and prints the values of `count`.

break vs continue vs pass

Statement	Action
break	Exits the loop immediately.
continue	Skips the current iteration and moves to the next one.
pass	Does nothing; acts as a placeholder.

Key Points

- `pass` is a null statement that performs no action.
- It is used as a placeholder for future code.
- `pass` can be used with loops, functions, classes, and conditional statements.
- Unlike `break` and `continue`, `pass` does not affect the flow of the loop.
- It helps avoid syntax errors when a code block is required but not yet implemented.

Loop else Clause

In Python, a loop can have an `else clause`. The code inside the `else` block is executed **after the loop finishes normally**.

If the loop is terminated using the `break statement`, the `else` block is **not executed**.

The `else` clause can be used with both `for` and `while` loops.

Syntax with for Loop

```
for variable in sequence:  
    # Code  
else:  
    # Code to execute after the loop finishes
```

Syntax with while Loop

```
while condition:  
    # Code  
else:  
    # Code to execute after the loop finishes
```

Example 1: for Loop with else

```
In [ ]: for i in range(1, 6):  
        print(i)  
        else:  
            print("Loop Finished")
```

Output

```
1  
2  
3  
4  
5  
Loop Finished
```

Explanation

The `for` loop completes all its iterations normally, so the `else` block is executed.

Example 2: `for` Loop with `break`

```
In [ ]: for i in range(1, 6):  
        if i == 3:  
            break  
        print(i)  
        else:  
            print("Loop Finished")
```

Output

```
1  
2
```

Explanation

The loop stops when `i` becomes `3` because of the `break` statement. Therefore, the `else` block is **not executed**.

Example 3: `while` Loop with `else`

```
In [ ]: count = 1  
  
while count <= 3:  
    print(count)  
    count += 1  
else:  
    print("Loop Finished")
```

Output

```
1  
2  
3  
Loop Finished
```

Explanation

The `while` loop ends normally after the condition becomes `False`, so the `else` block is executed.

Example 4: `while` Loop with `break`

```
In [ ]: count = 1

while count <= 5:
    if count == 4:
        break
    print(count)
    count += 1
else:
    print("Loop Finished")
```

Output

```
1
2
3
```

Explanation

The `break` statement terminates the loop before it finishes normally. Therefore, the `else` block is skipped.

When to Use the `else` Clause

Use the `else` clause when you want to execute code **only if the loop completes without being interrupted by `break`**.

Common uses include:

- Searching for an item in a list.
 - Checking whether a value exists.
 - Running a task after successful loop completion.
-

Key Points

- The `else` clause can be used with both `for` and `while` loops.
- The `else` block executes only if the loop finishes normally.
- If the loop is terminated using `break`, the `else` block is skipped.
- The `else` clause is useful for actions that should occur only after successful loop completion.

Common Errors

- Forgetting to update the loop variable in a `while` loop, causing an infinite loop.
- Incorrect indentation inside the loop.
- Forgetting that `range()` excludes the stop value.
- Confusing `break` with `continue`.
- Using the wrong loop condition.

Best Practices

- Use a `for` loop when the number of iterations is known.
- Use a `while` loop when the number of iterations is unknown.
- Update the loop variable in `while` loops.
- Keep loop logic simple and readable.
- Avoid unnecessary infinite loops.
- Use `break` and `continue` only when needed.
- Test your loops with different input values.

Coding Questions (10)

Solve the following programs to understand how loops work.

1. Write a Python program to print numbers from **1 to 10** using a `for` loop.
2. Write a Python program to print numbers from **10 to 1** using a `while` loop.
3. Write a Python program to print the multiplication table of a given number.
4. Write a Python program to calculate the sum of the first **N** natural numbers.
5. Write a Python program to print all even numbers between **1 and 50**.
6. Write a Python program to print all odd numbers between **1 and 50**.
7. Write a Python program to print the square of numbers from **1 to 10**.
8. Write a Python program to print each character of a string using a `for` loop.
9. Write a Python program to print the following pattern:

```
*
**
***
****
*****
```

10. Write a Python program to stop printing numbers when the value becomes **7** using the `break` statement.

Practice Programs (10)

Try solving these programs on your own before checking the solutions.

1. Write a Python program to count from **1 to 20** using a `while` loop.
2. Write a Python program to print numbers from **1 to 100** that are divisible by **5**.
3. Write a Python program to calculate the factorial of a number using a loop.
4. Write a Python program to count the number of digits in a given number.
5. Write a Python program to reverse a given number using a `while` loop.
6. Write a Python program to print the following pattern:

```
*****
****
***
**
*
```

7. Write a Python program to skip the number **5** while printing numbers from **1 to 10** using the `continue` statement.
8. Write a Python program to print the multiplication tables from **1 to 5** using nested loops.
9. Write a Python program to print the following number pattern:

```
1
12
123
1234
12345
```

10. Write a Python program to repeatedly ask the user to enter a number until they enter **0**, then terminate the loop.

Cheat Sheet

Loops

Loop	Description
<code>for</code>	Repeats code over a sequence or a fixed number of times
<code>while</code>	Repeats code while a condition is <code>True</code>

`for` Loop Syntax

```
for variable in sequence:  
    # Code
```

`while` Loop Syntax

```
while condition:  
    # Code
```

range() Function

Function	Description
<code>range(stop)</code>	Starts from <code>0</code> and stops before <code>stop</code>
<code>range(start, stop)</code>	Starts from <code>start</code> and stops before <code>stop</code>
<code>range(start, stop, step)</code>	Starts from <code>start</code> , changes by <code>step</code> , and stops before <code>stop</code>

Loop Control Statements

Statement	Purpose
<code>break</code>	Exits the loop immediately
<code>continue</code>	Skips the current iteration
<code>pass</code>	Does nothing (placeholder)
<code>else</code>	Executes after the loop finishes normally

Quick Tips

- Use `for` when the number of iterations is known.
- Use `while` when the condition controls the loop.
- Update the loop variable in a `while` loop.
- Remember: `range()` excludes the stop value.

Summary

In this lesson, you learned:

- What loops are and why they are used.
- The `for` and `while` loops.
- The `range()` function.

- Nested loops.
 - Infinite loops.
 - Loop control statements: `break` , `continue` , and `pass` .
 - The `else` clause with loops.
 - Common errors and best practices.
-

What's Next?

In **Part 6**, you will learn about **Functions**, including:

- Creating Functions
- Function Parameters
- Return Statement
- Scope of Variables
- Built-in Functions

Happy Coding! 🚀